

Adam Optimization Demystified: Enhancing Multiclass MLP Performance

Loghman Samani

29.April.2024

Abstract

In this article, we delve into the intricacies of Adam optimization, a powerful algorithm for enhancing the performance of multiclass multilayer perceptron, MLP, models. We explore its mathematical underpinnings, discussing how it optimizes the learning process by dynamically adjusting the learning rate based on past gradients. Through a step-by-step elucidation, we illustrate how Adam complements the training of MLPs, facilitating faster convergence and mitigating issues such as overfitting. Additionally, we elucidate the significance of techniques like learning rate decay, mini-batch gradient descent, and one-hot encoding in conjunction with Adam optimization to bolster the robustness and efficiency of multiclass MLPs. Through mathematical explanations and code snippets, this article aims to demystify Adam optimization, providing a comprehensive understanding of its role in enhancing the performance of multiclass MLP models.

Introduction

When endeavoring to construct a powerful neural network, numerous critical questions arise, demanding thorough analysis and answers to enhance efficiency and, consequently, the accuracy of the model in predicting new, unknown/unlabeled data. The foremost consideration is determining the most suitable model for training on our data. This decision hinges on the

nature of the data and the objectives of the model. For instance, if the dataset comprises images, text, audio, etc., it's essential to ascertain which types of neural networks are best suited for our data and what specific questions our model should address. Should the predictions be binary (yes or no), or should they entail more nuanced categorizations? Once the appropriate model is selected, attention turns to finer details, mostly revolving around the model's structure and our computational resources. Questions arise: Which optimizer is optimal? What hyperparameters yield the best results? Is regularization necessary to refine the training process and enhance inference? These are just a few among many inquiries that must be carefully addressed to optimize project efficiency.

In this article, we aim to elucidate the structure of a Multi-Layer Perceptron (MLP) model step by step and implement it from scratch in Python, utilizing the NumPy library for multiclass classification tasks. We will employ the Adam optimization algorithm as our optimization algorithm, with a special focus on this topic. Additionally, we will integrate L2 regularization into our model as a regularization technique. Another technique we will employ to enhance our model is learning rate decay, which accelerates the convergence process. Finally, to improve computational efficiency, we will utilize mini-batches for training the model.

Forward Propagation

The process of learning a supervised machine learning model relies on the model's predictions for input data, which are then compared to the actual labels. Parameters are adjusted iteratively to minimize the disparity between predicted and actual labels. The initial step in this iterative process is making predictions with the model, constituting the main aspect of forward propagation in a machine learning model, in our case an MLP model. During forward propagation, two primary learnable parameters are involved: weight, w , and bias, b , which are optimized during the training process. Mathematically, forward propagation can be explained as follows:

$$z_{j,i}^{[l]} = w_{j,k}^{[l]} \cdot a_{k,i}^{[l-1]} + b_j^{[l]}$$

In this equation, $z^{[l]}_{j,i}$ corresponds to the j -th node in the l -th layer, calculating the output for the i -th datapoint in the dataset. Here, k represents the number of nodes in the previous layer, indicating that the calculation in the current node is the sum of the multiplications between the i -th activation of the previous layer, $a^{l-1}_{k,i}$, and the corresponding weight, $w^{[l]}_{j,i}$, connecting layer $l - 1$ with the current layer l , along with the bias term, $b^{[l]}_j$, of the current layer.

Activation of the current node is then performed using an activation function, such as the rectified linear unit (ReLU), one of the most commonly used activation functions in hidden layers. This activation introduces non-linearity to the model. Without an activation function in the hidden layers, regardless of the network's depth, the model behaves similarly to a linear regression model.

$$a^{[l]}_{j,i} = g^{[l]} \left(z^{[l]}_{1,i}, \dots, z^{[l]}_{j,i}, \dots, z^{[l]}_{n^{[l]},i} \right)$$

Now, understanding the forward propagation process, it's time to implement it in Python code.

```
def forward_propagation(X, parameters):

    A = X
    L = len(parameters) // 2
    caches = []

    for l in range(1, L):
        W = parameters[f"W{l}"]
        b = parameters[f"b{l}"]
        A_prev = A
        linear_cache = (A_prev, W, b)

        Z = np.dot(W, A_prev) + b
        A, activation_cache = self.relu(Z=Z)
```

```

        caches.append((linear_cache , activation_cache))

W = parameters[f"W{L}"]
b = parameters[f"b{L}"]
A_prev = A
linear_cache = (A_prev , W, b)

Z = np.dot(W, A_prev) + b
A, activation_cache = self.softmax(Z=Z)

caches.append((linear_cache , activation_cache))

return A, caches

```

Rectified Linear Unit (ReLU)

The activation function we will use to activate the output of the forward equation in the hidden layers of the network is the Rectified Linear Unit (ReLU). While there are various activation functions suitable for hidden layers in a neural network, including Tanh, Sigmoid, ReLU, and Leaky ReLU, we opt for ReLU due to its efficiency compared to Sigmoid and Tanh. While Leaky ReLU is another option, we find that in our case, using ReLU suffices, as the differences between the two are not substantial and do not significantly impact the model we will build. The ReLU activation function replaces negative z values with zero, enabling the model to detect non-linearity in the training data. Mathematically, ReLU can be expressed as:

$$\begin{aligned}
 a_{j,i}^{[l]} &= g^{[l]} \left(z_{1,i}^{[l]}, \dots, z_{j,i}^{[l]}, \dots, z_{n^{[l]},i}^{[l]} \right) \\
 &= \max \left(z_{1,i}^{[l]}, \dots, z_{j,i}^{[l]}, \dots, z_{n^{[l]},i}^{[l]} \right)
 \end{aligned}$$

Its piecewise definition is as follows:

$$f(z) = \begin{cases} 0 & \text{if } z < 0 \\ z & \text{otherwise} \end{cases}$$

The Python implementation of ReLU is as follows:

```
def relu(Z):  
  
    A = np.maximum(0, Z)  
    activation_cache = Z  
  
    return (A, activation_cache)
```

SoftMax Activation Function

The choice of activation function for the last layer of the neural network depends on the training data and, more specifically, the purpose of the model. While choosing the appropriate activation function for hidden layers is not specifically dependent on the nature of the data or the model's purpose, selecting the activation function for the last layer is crucial. In the case of multiclass classification tasks the most appropriate choice is the SoftMax activation function. This function returns a set of floating-point numbers between 0 and 1 for each input z , interpreted as the probabilities of each example belonging to each of the classes. Mathematically, the SoftMax activation function is described as follows:

$$\begin{aligned} a_{j,i}^{[l]} &= g^{[l]} \left(z_{1,i}^{[l]}, \dots, z_{j,i}^{[l]}, \dots, z_{n^{[l]},i}^{[l]} \right) \\ &= \frac{\exp \left(z_{j,i}^{[l]} \right)}{\sum_p \exp \left(z_{p,i}^{[l]} \right)} \end{aligned}$$

The Python implementation of the SoftMax activation function is as follows:

```

def softmax(Z):

    A = np.divide(np.exp(Z) , np.sum(np.exp(Z) , axis=0,
        keepdims=True) + 1e-15)
    activation_cache = Z

    return (A, activation_cache)

```

Cost Function

In each iteration of the model, the data will be passed through the entire network. In each hidden layer of the network, the $Z^{[l]}$ array will be calculated using the parameters, $W^{[l]}$ and $b^{[l]}$, and the activated matrix of the previous layer $A^{[l-1]}$. This array will then be activated using the ReLU activation function, as explained in detail and implemented in Python. This process is repeated for all layers except the last layer, where the calculated $Z^{[L]}$ array will be activated using the SoftMax activation function, creating a matrix, $A^{[L]}$, of probabilities. Each column of the matrix corresponds to an example and each row corresponds to a class. Next, the loss of the data will be calculated based on the calculated probabilities belonging to each class for each example and the actual class of each data. The cost function used to compute the differences between predicted and actual labels is a specific version of the logarithmic loss entropy function, originally used to calculate the cost in a binary system. Mathematically, this specific cost function is expressed as follows:

$$\begin{aligned}
 J &= f(\hat{Y}, Y) = f(A^{[L]}, Y) \\
 &= \frac{1}{m} \sum_i \sum_j y_{j,i} \log(\hat{y}_{j,i}) \\
 &= \frac{1}{m} \sum_i \sum_j y_{j,i} \log(a_{j,i}^L)
 \end{aligned}$$

We can implement the cost function with the following Python code:

```

def compute_cost(parameters, A, Y, lambda_):

    L = len(parameters) // 2
    m = Y.shape[1]

    A = np.clip(A, 1e-15, 1 - 1e-15)
    loss = -np.sum(Y * np.log(A))
    L2_loss = (lambda_ / (2 * m)) * sum([np.sum(np.
        square(parameters[f"W{l}"])) for l in range(1, L
        + 1)])
    cost = np.squeeze(loss) + np.squeeze(L2_loss)

    return cost

```

Backward Propagation

After calculating the cost of the data, it serves as a guide for optimizing the learnable parameters, W and b . But how exactly does this optimization occur?

In the case of differentiable equations, such as the cost function (in our case, cross-entropy loss), the preferred approach is to compute the derivative of the cost function with respect to the learnable parameters and update the parameters based on the calculated gradients and a hyperparameter, α (learning rate). The primary optimization algorithm that utilizes derivatives to calculate gradients is the gradient descent algorithm. There are numerous optimization algorithms, many of which have gradient descent at their core, with their main calculations based on gradients. Examples include gradient descent with momentum, RMSprop, AdaGrad, AdaDelta, and Adam. In our model, we will implement and utilize the Adam algorithm as the optimization algorithm.

Adam Optimizer

In machine learning models, the objective is to find the set of optimal hyperparameters that minimize the cost function and optimize the learnable parameters efficiently. An important aspect of this process is the choice of

an appropriate optimizer capable of reducing the loss to an acceptable minimum level while considering time and computational resources. The Adam algorithm stands out as one of the most powerful optimizers, with widespread use in modern deep learning models.

Most deep learning models rely on the backpropagation algorithm, which propagates the error from the output layer backward and updates the learnable parameters layer by layer using gradient descent. Adam is a member of this family of optimizers, utilizing the backpropagation algorithm to minimize the loss.

```

Require:  $\alpha$ : Stepsize
Require:  $\beta_1, \beta_2 \in [0, 1)$ : Exponential decay rates for the moment estimates
Require:  $f(\theta)$ : Stochastic objective function with parameters  $\theta$ 
Require:  $\theta_0$ : Initial parameter vector
 $m_0 \leftarrow 0$  (Initialize 1st moment vector)
 $v_0 \leftarrow 0$  (Initialize 2nd moment vector)
 $t \leftarrow 0$  (Initialize timestep)
while  $\theta_t$  not converged do
   $t \leftarrow t + 1$ 
   $g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$  (Get gradients w.r.t. stochastic objective at timestep  $t$ )
   $m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$  (Update biased first moment estimate)
   $v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$  (Update biased second raw moment estimate)
   $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$  (Compute bias-corrected first moment estimate)
   $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$  (Compute bias-corrected second raw moment estimate)
   $\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$  (Update parameters)
end while
return  $\theta_t$  (Resulting parameters)

```

Figure 1: Adam algorithm. Good default settings for the tested machine learning problems are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$.

If we examine the original pseudocode description of the Adam algorithm depicted in Figure 1, we observe that the algorithm combines various techniques. It integrates gradient descent, first momentum, and second momentum to enhance its ability to traverse the optimization landscape effectively. Essentially, Adam combines mini-batch/stochastic gradient descent, momentum, and RMSprop algorithms. Leveraging the power of these three algorithms and incorporating a learning rate decay pattern makes Adam one of the fastest algorithms, capable of efficient convergence compared to many others.

Gradient Descent

In each iteration of the Adam algorithm, the first step is to calculate the gradients of the learnable parameters based on the loss function. This is accomplished using the chain rule in calculus, which enables the computation of partial derivatives of the loss with respect to the parameters. The initial step in this calculation involves computing the partial derivative of the loss function $J(\hat{y}, y)$ with respect to the activated output of the last layer, $A^{[L]}$. This matrix contains probabilities for each example belonging to each class. Subsequently, the partial derivative of the loss with respect to the output of the last layer, $\frac{\partial J}{\partial A^{[L]}}$, is utilized to compute $\frac{\partial J}{\partial Z^{[L]}}$, representing the derivative of the activation function of the last layer (in this case, the softmax activation function) with respect to $Z^{[L]}$. The final step is to calculate the partial derivatives of the loss function with respect to the parameters, $W^{[L]}$ and $b^{[L]}$. This process is repeated until the gradients of all learnable parameters of the entire model are calculated, necessitating traversal through the model from the last layer to the first layer. mathematically the entire process of gradient descent will be written as follows:

our goal is to calculate $\frac{\partial J}{\partial w}$ and $\frac{\partial J}{\partial b}$:

$$\begin{aligned}\frac{\partial J}{\partial w_{j,k}^{[l]}} &= \sum_i \frac{\partial J}{\partial z_{j,i}^{[l]}} \frac{\partial z_{j,i}^{[l]}}{\partial w_{j,k}^{[l]}} = \sum_i \frac{\partial J}{\partial z_{j,i}^{[l]}} a_{k,i}^{[l-1]} \\ \frac{\partial J}{\partial b_j^{[l]}} &= \sum_i \frac{\partial J}{\partial z_{j,i}^{[l]}} \frac{\partial z_{j,i}^{[l]}}{\partial b_j^{[l]}} = \sum_i \frac{\partial J}{\partial z_{j,i}^{[l]}}\end{aligned}$$

The $\frac{\partial J}{\partial z_{j,i}^{[l]}}$ can be calculated as follows:

$$\frac{\partial J}{\partial z_{j,i}^{[l]}} = \sum_p \frac{\partial J}{\partial a_{p,i}^{[l]}} \frac{\partial a_{p,i}^{[l]}}{\partial z_{j,i}^{[l]}}$$

for the output layer:

$$\begin{aligned}
&= \frac{\partial J}{\partial a_{j,i}^{[l]}} \frac{\partial a_{j,i}^{[l]}}{\partial z_{j,i}^{[l]}} + \sum_{p \neq j} \frac{\partial J}{\partial a_{p,i}^{[l]}} \frac{\partial a_{p,i}^{[l]}}{\partial z_{j,i}^{[l]}} \\
&= \frac{\partial J}{\partial a_{j,i}^{[l]}} a_{j,i}^{[l]} (1 - a_{j,i}^{[l]}) - \sum_{p \neq j} \frac{\partial J}{\partial a_{p,i}^{[l]}} a_{p,i}^{[l]} a_{j,i}^{[l]} \\
&= a_{j,i}^{[l]} \left(\frac{\partial J}{\partial a_{j,i}^{[l]}} (1 - a_{j,i}^{[l]}) - \sum_{p \neq j} \frac{\partial J}{\partial a_{p,i}^{[l]}} a_{p,i}^{[l]} \right) \\
&= a_{j,i}^{[l]} \left(\frac{\partial J}{\partial a_{j,i}^{[l]}} (1 - a_{j,i}^{[l]}) - \sum_p \frac{\partial J}{\partial a_{p,i}^{[l]}} a_{p,i}^{[l]} + \frac{\partial J}{\partial a_{j,i}^{[l]}} a_{j,i}^{[l]} \right) \\
&= a_{j,i}^{[l]} \left(\frac{\partial J}{\partial a_{j,i}^{[l]}} - \sum_p \frac{\partial J}{\partial a_{p,i}^{[l]}} a_{p,i}^{[l]} \right)
\end{aligned}$$

for the hidden layers:

$$\begin{aligned}
&= \frac{\partial J}{\partial a_{j,i}^{[l]}} \frac{\partial a_{j,i}^{[l]}}{\partial z_{j,i}^{[l]}} + \sum_{p \neq j} \frac{\partial J}{\partial a_{p,i}^{[l]}} \frac{\partial a_{p,i}^{[l]}}{\partial z_{j,i}^{[l]}} \\
&= \frac{\partial J}{\partial a_{j,i}^{[l]}} I(z_{j,i}^{[l]} > 0)
\end{aligned}$$

The Python implementation of gradient descent is as follows:

```

def backward_relu(dA, Z):

    dZ = np.array(dA, copy=True)
    dZ[Z <= 0] = 0

```

```

    return dZ

def backward_softmax(dAL, Z):

    A = np.divide(np.exp(Z), np.sum(np.exp(Z), axis=0,
        keepdims=True) + 1e-15)
    dZ = A * (1 - A) * dAL

    return dZ

def gradient_descent(AL, Y, caches, lambda_):

    Y = Y.reshape(AL.shape)
    L = len(caches)
    gradients = {}

    dAL = - (np.divide(Y, AL + 1e-15) - np.divide(1 - Y
        , 1 - AL + 1e-15))

    (linear_cache, activation_cache) = caches[-1]
    (A_prev, W, b) = linear_cache
    m = A_prev.shape[1]

    dZ = self.backward_softmax(dAL=dAL, Z=
        activation_cache)

    gradients[f"dW{L}"] = 1. / m * np.dot(dZ, A_prev.T)
        + (lambda_ / m) * W
    gradients[f"db{L}"] = np.sum(dZ, axis=1, keepdims=
        True) / m
    gradients[f"dA{L-1}"] = np.dot(W.T, dZ)

    for l in reversed(range(L - 1)):

        (linear_cache, activation_cache) = caches[l]
        (A_prev, W, b) = linear_cache
        m = A_prev.shape[1]

```

```

dZ = self.backward_relu(dA=gradients[f"dA{l+1}"+
1}"], Z=activation_cache)

gradients[f"dW{l+1}"] = 1. / m * np.dot(dZ,
A_prev.T) + (lambda_ / m) * W
gradients[f"db{l+1}"] = np.sum(dZ, axis=1,
keepdims=True) / m
gradients[f"dA{l}"] = np.dot(W.T, dZ)

return gradients

```

Note that we have defined three functions in the code block above to implement the gradient descent algorithm. The main function uses auxiliary functions to calculate the gradients of the loss function with respect to the parameters.

First Momentum

In the standard gradient descent algorithm, parameters W and b are updated based solely on the gradients computed in the current epoch:

$$w = w - \alpha \frac{\partial J}{\partial w}$$

$$b = b - \alpha \frac{\partial J}{\partial b}$$

However, momentum-based algorithms take into account gradients from previous steps, accelerating the learning process and facilitating faster convergence compared to basic gradient descent.

$$v\partial w = \beta.v\partial w + (1 - \beta) \partial w$$

$$v\partial b = \beta.v\partial b + (1 - \beta) \partial b$$

Here, β is the momentum hyperparameter, typically set to 0.9 by default. After computing $v_{\partial\theta}$ (where θ represents learnable parameters), bias correction is applied. While bias correction is commonly associated with Adam optimization, it is not necessary for gradient descent with momentum. Bias correction is essential in Adam because it helps adjust the moving averages during the early iterations of training.

Bias correction is defined as follows:

$$v\partial w_{corrected} = \frac{v\partial w}{(1 - \beta^t)}$$

$$v\partial b_{corrected} = \frac{v\partial b}{(1 - \beta^t)}$$

Here, t denotes the number of iterations in the training process.

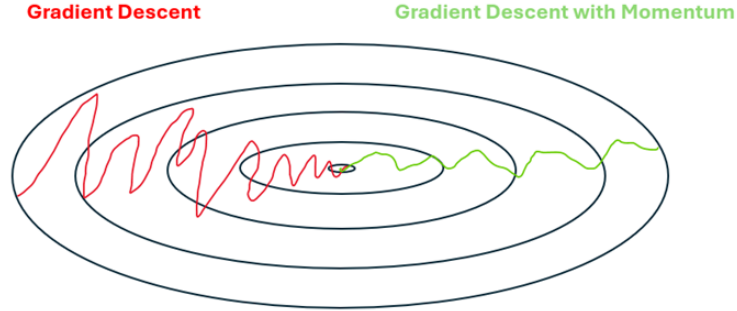


Figure 2: Compare gradient descent algorithm with and without momentum.

Figure 2 illustrates how momentum can expedite the gradient descent algorithm. Mathematically, momentum can be calculated as follows:

The mathematical equations above can be combined and implemented as shown in the Python function below:

```
def compute_first_momentum(parameters , gradients , V,
    beta , t):

    L = len(parameters) // 2

    for l in range(1, L + 1):

        V[f"vdW{l}"] = (beta * V[f"vdW{l}"]) + ((1 -
            beta) * gradients[f"dW{l}"])

        V[f"vdb{l}"] = (beta * V[f"vdb{l}"]) + ((1 -
            beta) * gradients[f"db{l}"])

        V[f"vdW{l}"] = V[f"vdW{l}"] / (1 - np.power(beta
            , t))

        V[f"vdb{l}"] = V[f"vdb{l}"] / (1 - np.power(beta
            , t))

    return V
```

Second Momentum

Adaptive gradient-based learning algorithms constitute a group of techniques that employ adaptive learning rates to update variables. These algorithms allow the learning rate to adapt based on the parameters, enabling significant updates for infrequent parameters and smaller updates for frequent parameters. This approach assigns a different learning rate to each parameter at a time step, leveraging past gradients computed for that parameter. RMSPprop is a member of this group, utilizing a moving average of the squared partial derivatives. In this technique, the second moment of the gradient is

calculated as follows:

$$\begin{aligned} s\partial w &= \beta \cdot s\partial w + (1 - \beta) \partial w^2 \\ s\partial b &= \beta \cdot s\partial b + (1 - \beta) \partial b^2 \end{aligned}$$

Similar to the modification made for the first momentum with bias correction terms, the second momentum is adjusted as follows:

$$\begin{aligned} s\partial w_{corrected} &= \frac{s\partial w}{(1 - \beta^t)} \\ s\partial b_{corrected} &= \frac{s\partial b}{(1 - \beta^t)} \end{aligned}$$

Here, t represents the number of iterations in the training process.

The Python function below demonstrates how to implement the second momentum:

```
def compute_second_momentum(parameters, gradients, S,
                             beta, t):

    L = len(parameters) // 2

    for l in range(1, L + 1):

        S[f"sdW{l}"] = (beta * S[f"sdW{l}"]) + ((1 -
            beta) * np.power(gradients[f"dW{l}"], 2))
```

```

S[f"sdb{1}"] = (beta * S[f"sdb{1}"]) + ((1 -
    beta) * np.power(gradients[f"db{1}"], 2))

S[f"sdW{1}"] = S[f"sdW{1}"] / (1 - np.power(
    beta, t))

S[f"sdb{1}"] = S[f"sdb{1}"] / (1 - np.power(
    beta, t))

return S

```

Update Parameters

After calculating the first and second momentum using gradients, the parameters are updated as follows:

$$W^{[l]} = W^{[l]} - \alpha \frac{v\partial W^{[l]}}{\sqrt{s\partial W^{[l]} + \epsilon}}$$

$$b^{[l]} = b^{[l]} - \alpha \frac{v\partial b^{[l]}}{\sqrt{s\partial b^{[l]} + \epsilon}}$$

Here, α represents the learning rate, and ϵ is a small number to prevent zero division (a recommended value for ϵ is 10^{-8}).

We can implement the update parameters function as follows:

```

def update_parameters(parameters, V, S, learning_rate,
    epsilon):

    L = len(parameters) // 2

    for l in range(1, L + 1):

```



```

parameters[f"W{l}"] -= learning_rate * (V[f"vdW
{l}"] / np.sqrt(S[f"sdW{l}"] + epsilon))
parameters[f"b{l}"] -= learning_rate * (V[f
"vdb{l}"] / np.sqrt(S[f"sdb{l}"] +
epsilon))

return parameters

```

L2 Regularization Technique

Machine learning models, especially neural networks, are powerful tools capable of recognizing complex patterns in training data and adapting the model to predict new examples accurately. However, this adaptability can sometimes lead to overfitting, where the model performs poorly on unseen data. To address this issue, various regularization techniques are employed, such as L1 (Lasso) and L2 (Ridge) regularization, dropout, and increasing the size of the training dataset. In our model, we utilize L2 regularization to prevent overfitting. L2 regularization adds the sum of the squared values of the model's parameters to the cost function, encouraging smaller but non-zero coefficients. Mathematically, it is defined as:

$$L_2 = ||w||^2 = \sum_i \sum_j (w_{i,j})^2$$

The regularization term is added to both the cost function and the backward propagation step as follows:

Cross-entropy loss:

$$J(A^{[L]}, Y) = \frac{1}{m} \sum_i \sum_j y_{j,i} \log(a_{j,i}^{[L]}) + \frac{\lambda}{2m} L^2$$

Gradient:

$$\frac{\partial J}{\partial w_{j,k}^{[l]}} = \sum_i \frac{\partial J}{\partial z_{j,i}^{[l]}} a_{k,i}^{[l-1]} + \frac{\lambda}{m} w_{j,k}^{[l]}$$

It's important to note that the regularization term is only added to the gradient of weights, $\frac{\partial J}{\partial w}$, and there is no need to include it in the gradient of biases, $\frac{\partial J}{\partial b}$.

Learning Rate Decay

As we know, the learning process can slow down during training due to the reduction of slope along the downhill path of the learning curve. It would be beneficial if the step sizes could automatically decrease during the training process. One powerful technique for achieving this is learning rate decay, where the learning rate is gradually reduced over time. As shown in Figure 3, learning rate decay techniques are useful as they allow the model to converge more efficiently.



Figure 3: Compare learning process with and without learning rate decay technique

There are different learning rate decay techniques to reduce the learning rate during training. One approach, which we will use in our model, is to discretely reduce the learning rate at intervals based on the number of epochs. The mathematical equation for this technique is as follows:

$$\alpha_t = \frac{\alpha_0}{1 + \text{decayrate} \cdot \left(\lceil \frac{\text{number of epochs}}{\text{time interval}} \rceil\right)}$$

Here, the decay rate is a hyperparameter that needs to be tuned. The implementation of this technique is as follows:

```
def update_learning_rate(init_learning_rate , epoch_num ,
    decay_rate , time_interval):

    learning_rate = init_learning_rate / (1 +
        decay_rate * (np.floor(epoch_num / time_interval
        )))

    return learning_rate
```

Mini-Batch Gradient Descent

One of the most widely used and popular training techniques in machine learning projects, especially in deep learning projects, is to divide the training set into randomly chosen subsets. This technique is called mini-batch gradient descent. The training process with mini-batch gradient descent proceeds as follows: in each epoch of the training process, the training set is divided into subsets, each containing a constant number of examples. Common subset sizes are 16, 32, 128, 256, 512. Then, each subset is passed through the network, and the learnable parameters are updated based only on the calculated gradients within that subset. After all subsets have been passed through the network, the next epoch begins, and the training dataset is again randomly divided into subsets containing a different set of examples. This ensures that the model is trained in each iteration with a different set of data.

This technique is particularly useful when the training data contains a large number of examples and the model is deep, as it helps reduce both training time and the risk of overfitting. The code below implements the

function that divides the training set, both examples X and labels Y , into subsets.

```
def random_mini_batches(X, Y, mini_batch_size, seed):

    np.random.seed(seed)
    m = X.shape[1]
    n = Y.shape[0]
    mini_batches = []

    permutation = np.random.permutation(m)
    shuffled_X = X[:, permutation]
    shuffled_Y = Y[:, permutation].reshape((n, m))

    # Step 2: Partition the shuffled data into mini-
        batches
    num_complete_minibatches = m // mini_batch_size

    for k in range(num_complete_minibatches):
        start_idx = k * mini_batch_size
        end_idx = (k + 1) * mini_batch_size
        mini_batch_X = shuffled_X[:, start_idx:end_idx]
        mini_batch_Y = shuffled_Y[:, start_idx:end_idx]
        mini_batches.append((mini_batch_X, mini_batch_Y
                               ))

    # Handling the end case (last mini-batch may have
        fewer examples)
    if m % mini_batch_size != 0:
        start_idx = num_complete_minibatches *
            mini_batch_size
        mini_batch_X = shuffled_X[:, start_idx:]
        mini_batch_Y = shuffled_Y[:, start_idx:]
        mini_batches.append((mini_batch_X, mini_batch_Y
                               ))

    return mini_batches
```

One-Hot Encoding

In the context of multiclass classification problems, the softmax activation function is commonly used to transform the output of the last layer of the network into probabilities for each example belonging to each class. This means that the $A^{[L]}$ array, which has the shape (1, number of examples), needs to be converted into a matrix with the shape (number of classes, number of examples). To compare the output probabilities to the actual labels, they must have the same size. Therefore, it is necessary to reshape the input labels to match the shape of the predicted probabilities. To achieve this, each label is placed in a column of a matrix with the number of rows equal to the number of classes. The corresponding row of the true label in each example is filled with a one, while the rest of the rows are filled with zeros. This results in each column containing only one 1 and the rest of the entries as zeros. The code below implements the one-hot encoding function. It is important to note that the input label array should have only one dimension (number of examples,).

```
def onehot_encoding(Y):  
  
    num_classes = len(np.unique(Y))  
    one_hot_Y = np.eye(num_classes)[Y].T  
  
    return one_hot_Y
```

Initialization Process

One of the critical aspects of building a neural network is the initialization of the matrices, including parameter matrices stored in a dictionary called parameters, first momentum matrices in a dictionary called first momentum, and second momentum matrices in a dictionary called second momentum. The general rule for constructing these matrices and vectors with the correct shape is as follows:

$$W_{(n^{[l]}, n^{[l-1]})}^{[l]}$$
$$b_{(n^{[l]}, n^{[l-1]})}^{[l]}$$

$$v\partial W_{(n^{[l]}, n^{[l-1]})}^{[l]}$$

$$v\partial b_{(n^{[l]}, n^{[l-1]})}^{[l]}$$

$$s\partial W_{(n^{[l]}, n^{[l-1]})}^{[l]}$$

$$s\partial b_{(n^{[l]}, n^{[l-1]})}^{[l]}$$

In the above formulas, each parameter matrix has a shape with the number of rows equal to the number of units in the corresponding layer and the number of columns equal to the number of units in the previous layer. The function below demonstrates how to initialize the parameter matrices using Xavier initialization for W matrices and zeros for bias matrices:

```
def initialize_adam_parameters(num_layer_units):

    parameters = {}
    first_moment = {}
    second_moment = {}
    L = len(num_layer_units)

    for l in range(1, L):

        parameters[f"W{l}"] = np.random.randn(
            num_layer_units[l], num_layer_units[l - 1])
            * np.sqrt(2 / num_layer_units[l - 1])
        parameters[f"b{l}"] = np.zeros((num_layer_units
            [l], 1))
        first_moment[f"vdW{l}"] = np.zeros((
            num_layer_units[l], num_layer_units[l - 1]))
        )
        first_moment[f"vdb{l}"] = np.zeros((
            num_layer_units[l], 1))
        second_moment[f"sdW{l}"] = np.zeros((
            num_layer_units[l], num_layer_units[l - 1]))
        )
```

```

second_moment[f"sdb{1}"] = np.zeros((
    num_layer_units[1], 1))

return (parameters, first_moment, second_moment
    )

```

The Xavier initialization method is used to initialize W matrices, while the rest of the parameters are initialized with zeros.

After all necessary functions have been created, we can now connect them with the function below, designed to call and sequence them appropriately. We've named this function "train."

Listing 1: Train the Model

```

def train(X_train, Y_train, num_hidden_units,
    num_epochs=1000, learning_rate=1e-3, lambda_=0.0,
    beta1=0.9, beta2=0.999, epsilon=1e-8, decay_rate
    =0.3, time_interval=100, minibatch_size=64, seed=1):

    costs = []
    Y_train = onehot_encoding(Y=Y_train)
    num_input_units = X_train.shape[0]
    num_output_units = Y_train.shape[0]
    num_layer_units = [num_input_units] + list(
        num_hidden_units) + [num_output_units]

    parameters, V, S = initialize_parameters(
        num_layer_units=num_layer_units)

    init_learning_rate = learning_rate

    epoch_num = 0
    m = X_train.shape[1]
    t = 0
    for i in range(num_epochs):

        seed = seed + 1
        mini_batches = random_mini_batches(

```

```

        X=X_train ,
        Y=Y_train ,
        mini_batch_size=minibatch_size ,
        seed=self.seed
    )
    cost_total = 0

    for minibatch in mini_batches:
        X_minibatch, Y_minibatch = minibatch
        A_minibatch, minibatch_caches =
            forward_propagation(
                X=X_minibatch ,
                parameters=parameters
            )
        cost_total += compute_cost(
            parameters=parameters ,
            A=A_minibatch ,
            Y=Y_minibatch ,
            lambda_=lambda_
        )
        gradients = backward_propagation(
            AL=A_minibatch ,
            Y=Y_minibatch ,
            caches=minibatch_caches ,
            lambda_=lambda_ ,
        )
        t += 1
        first_moment = compute_first_momentum(
            parameters=parameters ,
            gradients=gradients ,
            V=V ,
            beta=beta1 ,
            t=t
        )
        S = compute_squared_momentum(
            parameters=parameters ,
            gradients=gradients ,
            S=S ,

```



```

        beta=beta2 ,
        t=t
    )
    parameters = update_parameters(
        parameters=parameters ,
        V=V,
        S=S,
        learning_rate=learning_rate ,
        epsilon=epsilon
    )

    costs.append(cost_total / m)
    epoch_num += 1
    learning_rate = update_learning_rate(
        init_learning_rate=init_learning_rate ,
        epoch_num=epoch_num ,
        decay_rate=decay_rate ,
        time_interval=time_interval
    )
return parameters , costs

```

Conclusion

In this article, we delved into the intricacies of training multiclass classification Multilayer Perceptrons (MLPs). Beginning with an exploration of fundamental techniques like gradient descent and momentum-based optimization, we gradually expanded our understanding to include advanced concepts such as adaptive learning rate methods like RMSprop, Adam, L2 regularization, learning rate decay, mini-batch gradient descent, and one-hot encoding. Throughout our journey, we dissected each technique, providing mathematical formulations and practical Python implementations. From initializing parameters to orchestrating the training process, every aspect was meticulously covered, ensuring a comprehensive understanding for readers. We emphasized the importance of each technique in enhancing model performance, improving convergence speed, and mitigating common issues like overfitting. By applying these techniques judiciously, practitioners can effectively train multiclass classification MLPs, achieving higher accuracy and

robust generalization on unseen data.

In essence, this article serves as a roadmap for navigating the intricate landscape of training multiclass classification MLPs. Armed with a deep understanding of these techniques and their implementations, practitioners can confidently tackle real-world classification tasks, leveraging the power of neural networks to solve complex problems in various domains.

References

Adam: A Method for Stochastic Optimization

Improving the efficiency of RMSProp optimizer by utilizing Nestrovo in deep learning

Adaptive Subgradient Methods for Online Learning and Stochastic Optimization

Neural Network and Deep Learning

Improving Deep Neural Networks: Hyperparameter Tuning, Regularization and Optimization

My GitHub